

Topic 1 — Supervised Learning

Intelligent Loan Approval & Credit Risk Prediction System

Difficulty: Medium–Hard

Domain: Banking / FinTech / Machine Learning

ML Type: Supervised Learning — Classification

1. Project Description

Develop a full-stack machine learning application that predicts whether a loan application should be **Approved, Rejected, or flagged as High Risk** based on an applicant's financial and demographic information.

The system should train a classification model on historical loan data and then expose the trained model through a **Python backend + Streamlit frontend**.

The project should go beyond simply predicting approval. It should provide **risk analysis, probability scores, feature contribution/importance, and model performance metrics**, making it appropriate for a B.Tech final-year engineering project.

2. Problem Statement

Financial institutions receive thousands of loan applications and need to evaluate the applicant's creditworthiness efficiently.

Manual evaluation can be:

- Time-consuming
- Inconsistent
- Difficult to scale
- Prone to human error

The proposed system uses historical loan data to learn patterns associated with successful and risky applications and automatically predicts the outcome for a new applicant.

3. Machine Learning Component

Input Features

The model can use features such as:

Category	Example Features
Personal	Age, Education, Dependents
Employment	Employment Type, Years Employed
Income	Annual Income, Monthly Income
Credit	Credit Score, Credit History
Loan	Loan Amount, Loan Term
Financial	Existing Debt, Debt-to-Income Ratio
Assets	Property Value, Savings, Assets
Application	Previous Defaults, Number of Existing Loans

Target

Example:

None

Loan_Status

0 → Rejected

1 → Approved

An optional advanced formulation can classify applicants into:

None

Low Risk

Medium Risk

High Risk

4. Model Training

Students should experiment with multiple classification algorithms rather than training only one model.

Recommended Models

Baseline:

- Logistic Regression
- Decision Tree

Intermediate:

- Random Forest
- Gradient Boosting
- XGBoost / LightGBM

Advanced:

- Support Vector Machine
- CatBoost

The final model should be selected based on validation performance rather than simply choosing the most complex algorithm.

Model Comparison

The training pipeline should generate a comparison such as:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	—	—	—	—	—
Decision Tree	—	—	—	—	—
Random Forest	—	—	—	—	—
XGBoost	—	—	—	—	—

5. Data Preprocessing

This is an important part of the project.

Students should implement:

- Missing-value handling
- Duplicate detection
- Outlier analysis
- Numerical feature scaling
- Categorical encoding
- Feature selection
- Train/test split
- Class imbalance handling
- Data validation

Possible techniques:

None

`StandardScaler`

`MinMaxScaler`

`OneHotEncoder`

`LabelEncoder`

`SMOTE`

The preprocessing pipeline used during training **must be identical during prediction**.

6. Model Extraction

After training, the final production model must be exported.

Example:

None

`models/`

|

|— `loan_model.pkl`

|— `scaler.pkl`

|— `encoder.pkl`

|— `feature_columns.pkl`

Using `joblib` or `pickle`.

For example:

```
None  
loan_model.pkl
```

contains the trained ML model.

```
None  
scaler.pkl
```

contains the fitted scaler.

```
None  
encoder.pkl
```

contains the categorical encoding logic.

This prevents the application from retraining the model every time a user makes a prediction.

7. Backend — Python

The backend should contain the actual prediction logic.

Recommended architecture:

```
None  
User Input  
  ↓  
Validation  
  ↓  
Preprocessing  
  ↓  
Load scaler.pkl / encoder.pkl
```

```
↓  
Load loan_model.pkl  
↓  
Prediction  
↓  
Probability / Risk Score  
↓  
Response
```

The backend should provide functionality such as:

/predict

Receives applicant information and returns:

```
JSON  
{  
  "prediction": "Approved",  
  "approval_probability": 87.4,  
  "risk_level": "Low"  
}
```

/health

Returns whether the ML service is running.

Optional

```
None  
/model-info
```

Returns:

- Model name
- Model version
- Training date
- Accuracy
- F1 score

- ROC-AUC
-

8. Frontend — Streamlit

The Streamlit application should not be just a simple form.

Page 1 — Home

Display:

- Project title
- Problem statement
- Project objective
- ML workflow
- Technology stack

Page 2 — Loan Prediction

User enters:

```
None
Age
Income
Credit Score
Employment Type
Loan Amount
Loan Term
Existing Debt
Dependents
Education
Property Value
...
```

Then:

Predict Loan Eligibility

The application should display:

None

Prediction: APPROVED

Approval Probability: 87.4%

Risk Level: LOW

Page 3 — Applicant Risk Analysis

Display visualizations such as:

- Income vs Loan Amount
 - Credit Score distribution
 - Debt-to-Income ratio
 - Risk probability
 - Applicant financial profile
-

Page 4 — Model Performance

Display:

- Accuracy
 - Precision
 - Recall
 - F1-score
 - ROC-AUC
 - Confusion Matrix
 - ROC Curve
 - Feature Importance
-

9. Explainability

For a **medium-hard B.Tech project**, model explainability should be included.

The application should answer:

Why did the model approve/reject this applicant?

For example:

None

Major factors affecting prediction:

- + High Credit Score
- + Stable Employment
- + Low Debt-to-Income Ratio
- + Good Credit History
- High Loan Amount

Students can implement:

- Feature importance
- SHAP
- Permutation Importance

SHAP would make the project considerably stronger academically.

10. Required Project Structure

A recommended final structure:

None

```
loan-risk-prediction/
|
├── data/
|   └── loan_dataset.csv
|
├── notebooks/
|   └── EDA_and_Model_Training.ipynb
|
├── models/
|   ├── loan_model.pkl
|   ├── scaler.pkl
|   └── encoder.pkl
```

```
|   └─ feature_columns.pkl
|
├─ backend/
|   ├── app.py
|   ├── predictor.py
|   └─ preprocessing.py
|
├─ frontend/
|   └─ streamlit_app.py
|
├─ utils/
|   ├── validation.py
|   └─ visualization.py
|
├─ requirements.txt
├─ README.md
└─ .gitignore
```

11. requirements.txt

The project must have a proper dependency file.

For example:

```
None
pandas
numpy
scikit-learn
joblib
matplotlib
seaborn
streamlit
fastapi
uvicorn
```

```
shap
imbalanced-learn
```

If XGBoost is used:

```
None
xgboost
```

The project should be installable using:

```
Shell
pip install -r requirements.txt
```

12. Engineering Requirements

To make this appropriate for **B.Tech Final Year**, students should demonstrate separation between:

Training

```
None
train.py
  ↓
Dataset
  ↓
Preprocessing
  ↓
Model Training
  ↓
Evaluation
  ↓
Save Model
```

and:

Production Prediction

```
None
Streamlit
  ↓
Backend
  ↓
Load .pkl files
  ↓
Preprocess Input
  ↓
Prediction
  ↓
Result
```

The application must **NOT** retrain the model when making a prediction.

13. Evaluation

Students should report:

Classification Metrics

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

ML Engineering Evaluation

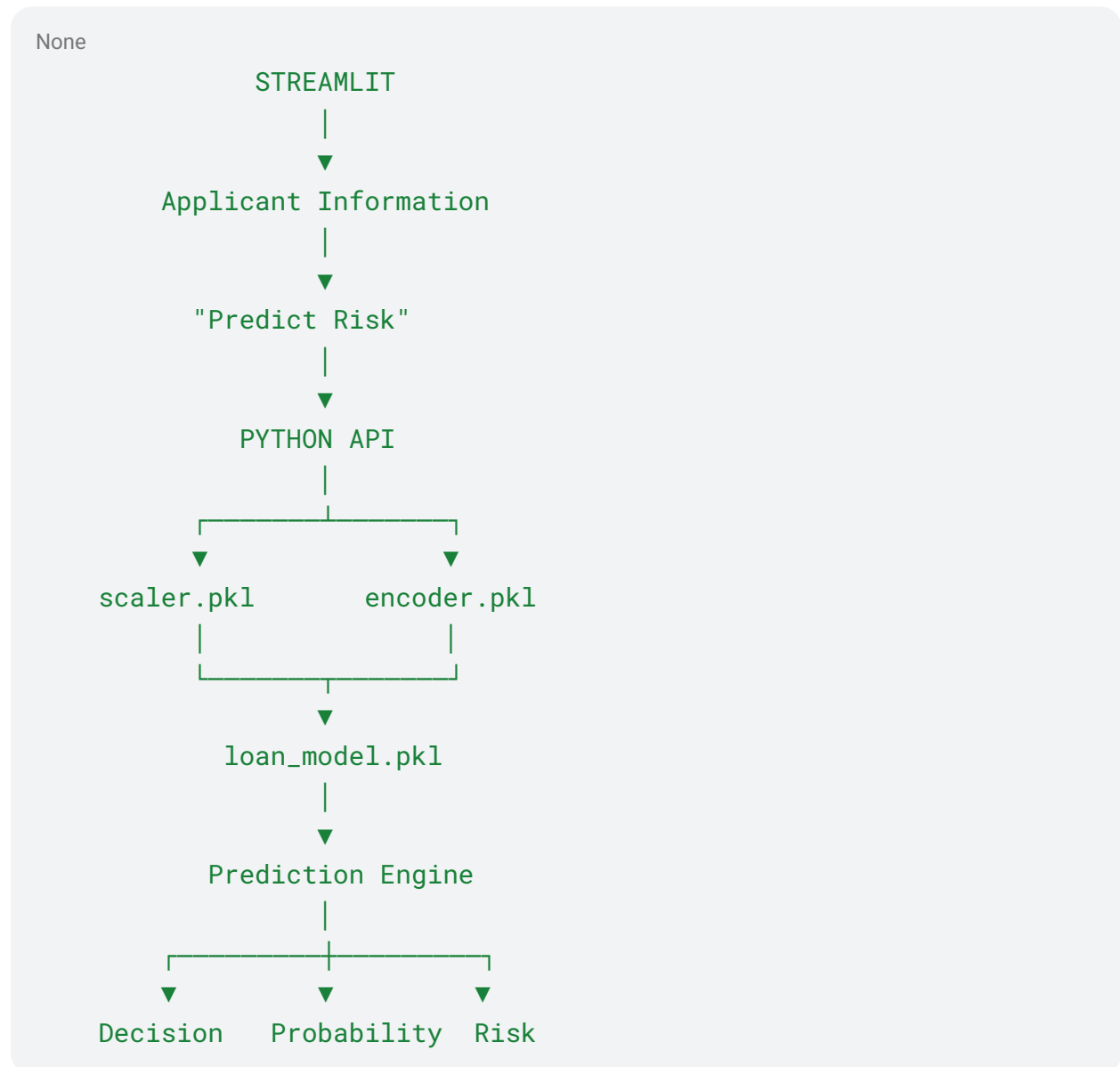
Also evaluate:

- Prediction latency
- Model size
- Preprocessing consistency
- Handling of invalid inputs
- Missing-value handling

- Class imbalance

14. Expected Final Demo

A good final demonstration would look like:



Final output

None

LOAN RISK ANALYSIS	
Decision	: APPROVED
Probability	: 87.4%
Risk Level	: LOW
Credit Score	: 782
DTI Ratio	: 28.3%
Key Factors	
✓ Strong Credit History	
✓ Stable Employment	
✓ Low Debt Ratio	